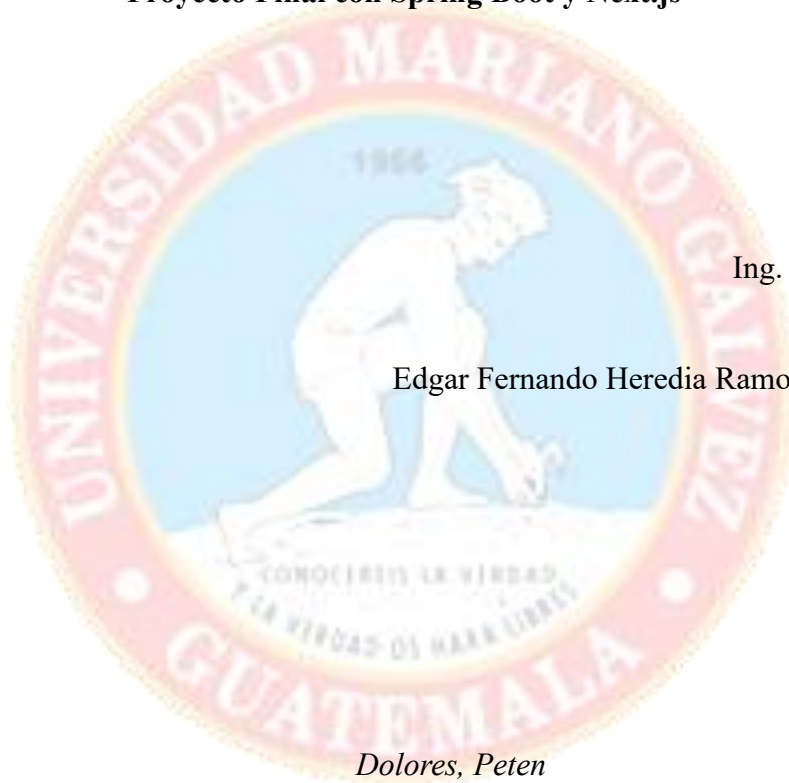


Universidad Mariano Gálvez de Guatemala
Ingeniería en Sistemas de Información
8vo. Semestre
Ciclo 2025

Tema:

Proyecto Final con Spring Boot y Next.js



Desarrollo Web

Ing. Walter Cordova

Edgar Fernando Heredia Ramos 1690-22-2199

Fecha:

15/11/2025

Dolores, Peten

INDICE

1	Introducción General	1
2	Objetivos del Proyecto	2
3	Marco Teorico.....	3
3.1	Resumen Ejecutivo del Proyecto.....	3
3.2	Componentes del Sistema.....	4
3.2.1	Componente A – Clientes y Pedidos (MariaDB).....	4
3.2.2	Componente B – Proveedores y Facturas (PostgreSQL).....	4
3.2.3	Componente C – Dependencia Maven común	4
3.2.4	Frontend – Next.js 14	5
3.3	Arquitectura General	5
3.4	DIAGRAMA 1 – Arquitectura general (PlantUML).....	6
3.5	DIAGRAMA 2 – Modelo de Datos (PlantUML)	8
3.6	DIAGRAMA 3 – Flujo de creación de Factura (Integración A -> B)	9
3.7	DIAGRAMA 4 - Flujo de Datos Completo: Backend → Frontend Next.js 10	
3.8	Documentación OpenAPI 3.....	11
3.9	Aplicación Web	15
3.9.1	Login.....	15
3.9.2	Dashboard.....	15
3.9.3	Componente Clientes.....	16
3.9.4	Componente Pedidos	16
3.9.5	Componente Proveedores	17
3.9.6	Componente Facturas	17
3.9.7	Componente Usuarios – Vista Admin	18

3.9.8	Componente Usuarios – VistaUser.....	18
4	Conclusiones.....	19



1 INTRODUCCIÓN GENERAL

Este proyecto integra backend, frontend, microservicios, bases de datos heterogéneas y un módulo Maven. El propósito es demostrar conocimientos avanzados adquiridos durante el curso, incluyendo:

- Arquitectura de software
- Desarrollo con Spring Boot 3 + Java 17
- Microservicios REST con OpenAPI 3
- Persistencia con JPA sobre MariaDB y PostgreSQL
- Frontend profesional con Next.js 14
- Documentación, pruebas y diseño modular
- Uso de un componente Maven reutilizable

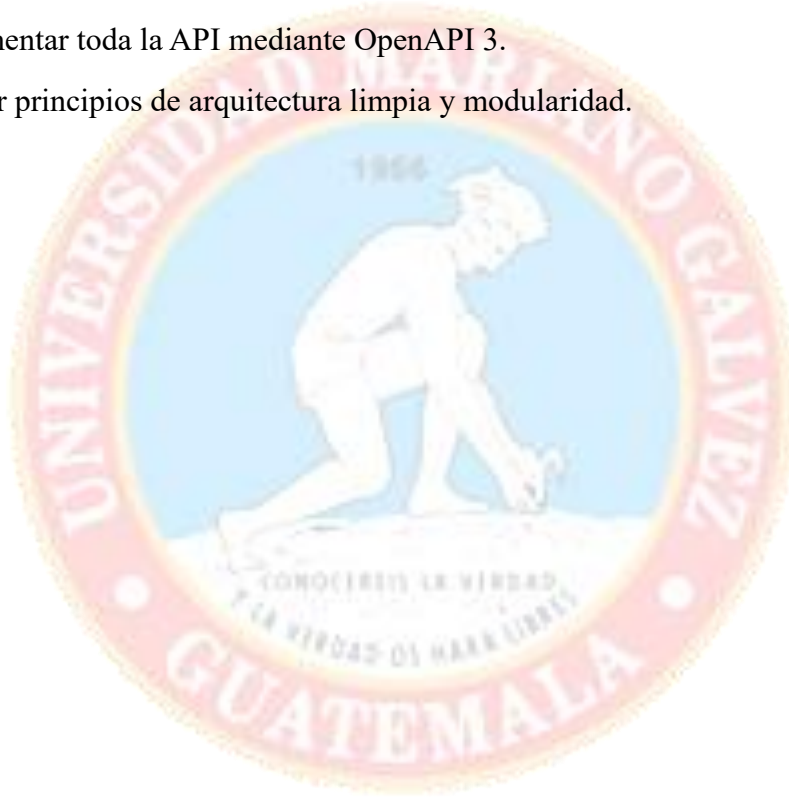
La solución simula un sistema de logística donde:

- Los clientes realizan pedidos (Componente A – MariaDB).
- Los proveedores emiten facturas (Componente B – PostgreSQL).
- Un módulo común (Componente C) contiene lógica compartida entre ambos.

Esta arquitectura demuestra habilidades técnicas, responsabilidad académica y alto desempeño.

2 OBJETIVOS DEL PROYECTO

1. Desarrollar dos microservicios completamente independientes y funcionales.
2. Integrar bases de datos distintas:
 - MariaDB (Clientes/Pedidos)
 - PostgreSQL (Proveedores/Facturas)
3. Implementar comunicación entre microservicios ($A \leftrightarrow B$).
4. Construir un frontend moderno que consuma ambos servicios.
5. Crear un componente Maven reutilizable.
6. Documentar toda la API mediante OpenAPI 3.
7. Aplicar principios de arquitectura limpia y modularidad.



3 MARCO TEORICO

3.1 Resumen Ejecutivo del Proyecto

Este proyecto es una solución empresarial que integra 3 tecnologías principales:

■ Backend (Microservicios con Spring Boot 3 + Java 17):

- A: Clientes y Pedidos (MariaDB)
- B: Proveedores y Facturas (PostgreSQL)
- C: Componente Maven común importado por A y B

■ Frontend (Next.js 14):

- Panel administrativo completo
- Manejo de estado reactivo
- Consumo de APIs REST de ambos microservicios

■ Persistencia Heterogénea:

- MariaDB (A)
- PostgreSQL (B)

■ Integración REST entre microservicios

- Componente B llama al A para obtener pedidos
- Componente B actualiza el estado de pedidos en A
- Comunicación vía cliente HTTP (PedidoClient)

■ Documentación con OpenAPI 3

- Generación automática de endpoints
- Modelos, parámetros, responses, schemas

Este sistema demuestra habilidades avanzadas en arquitectura, programación distribuida y diseño modular.

3.2 Componentes del Sistema

3.2.1 *Componente A – Clientes y Pedidos (MariaDB)*

Este microservicio maneja:

- Clientes
- Productos
- Pedidos
- Estados del pedido (PENDIENTE, FACTURADO, ANULADO)

Expone endpoints REST como:

- Crear y listar clientes
- Crear pedidos
- Listar pedidos pendientes por cliente
- Modificar estado del pedido
- Consultar pedidos por ID

Toda la API está documentada automáticamente con Swagger/OpenAPI.

3.2.2 *Componente B – Proveedores y Facturas (PostgreSQL)*

Este módulo administra:

- Proveedores
- Facturas
- Subtotal, IVA, descuentos y total final
- Asociación automática con pedidos del microservicio A

Incluye integración REST usando un cliente HTTP (PedidoClient) hacia el Componente A.

3.2.3 *Componente C – Dependencia Maven común*

El proyecto C incluye lógica reutilizable como:

- Cálculos de negocio (OperacionesNegocio)
- Validaciones
- Utilidades independientes del framework

Ambos microservicios A y B importan esta dependencia.

Esta separación mejora la mantenibilidad y evita duplicación de código.

3.2.4 Frontend – Next.js 14

El frontend unifica toda la funcionalidad:

- Dashboard para gestionar clientes, pedidos, proveedores y facturas
- Consumo de APIs REST
- Manejo de formularios y validaciones
- Modal de detalle de factura
- Estilo profesional con TailwindCSS

Incluye además un microservicio extra de autenticación (opcional), demostrando modularidad adicional.

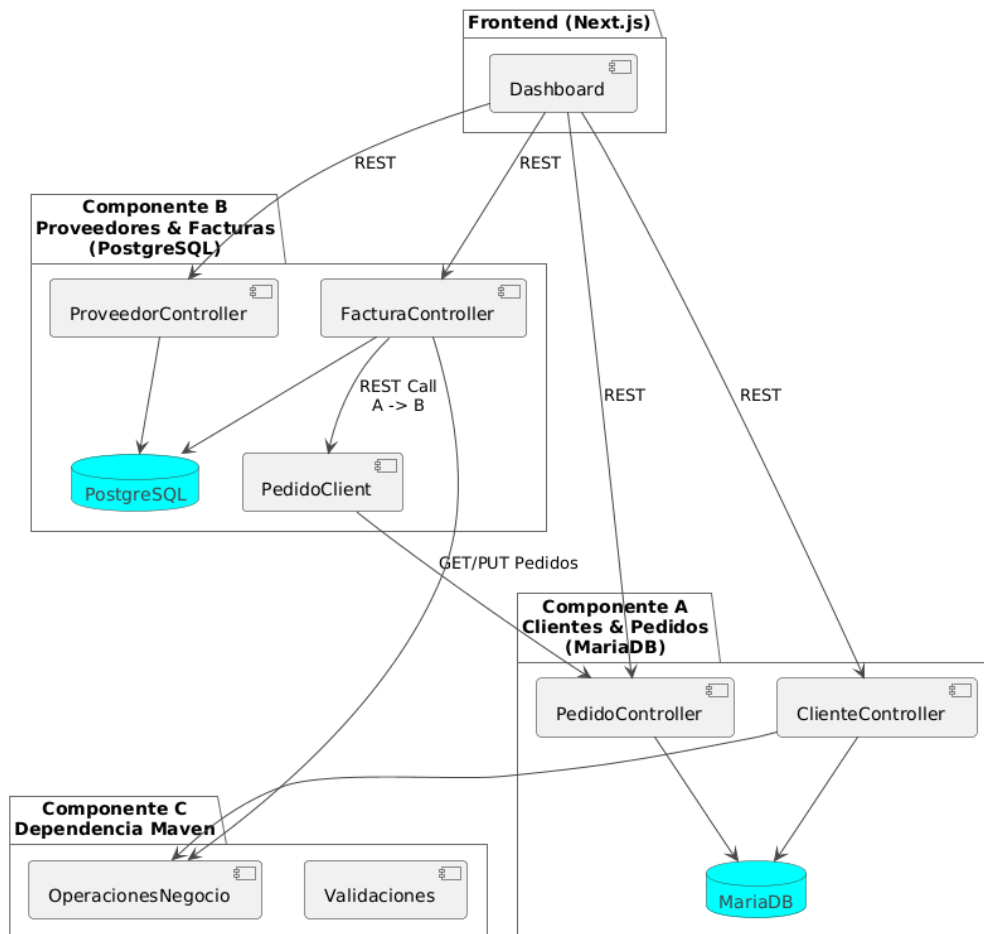
3.3 Arquitectura General

El sistema implementa una arquitectura limpia basada en microservicios:

- Comunicados vía HTTP (REST)
- Cada uno con su propia base de datos
- Un módulo común con lógica pura de negocio
- Un frontend desacoplado que consume ambos

A continuación, el diagrama de arquitectura en PlantUML:

3.4 DIAGRAMA 1 – Arquitectura general (PlantUML)



1. Frontend (Next.js)

- Opera como cliente independiente
- Consume endpoints del microservicio A y B
- Manejo reactivo con Hooks y Fetch API

2. Microservicio A

- Expuesto en `http://localhost:8080`
- Controladores REST → Pedidos y Clientes
- Persiste datos en MariaDB

3. Microservicio B

- Expuesto en `http://localhost:8081`

- Controladores de Facturas y Proveedores
- Usa PostgreSQL para persistencia

4. **Componente C**

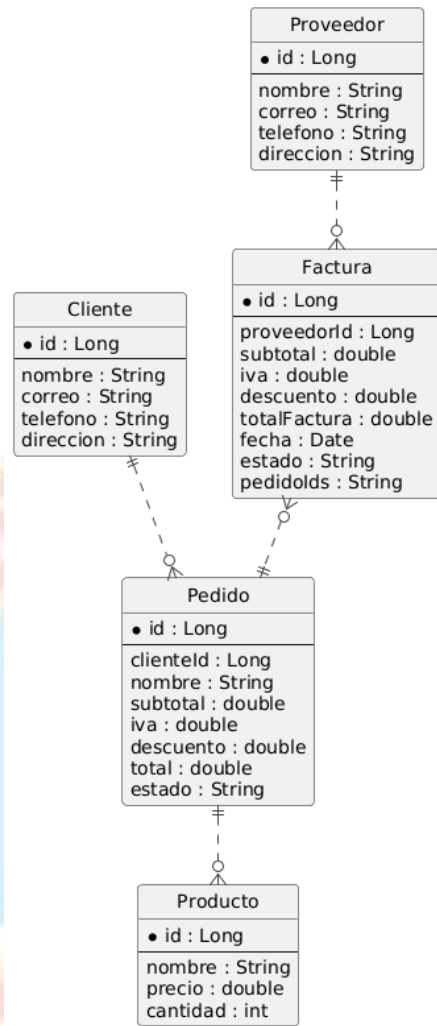
- Biblioteca Java usada por ambos servicios
- Lógica de negocio compartida (IVA, descuentos, cálculos)

5. **Integración A ↔ B**

- B llama a A para obtener pedidos y actualizar su estado
- Mantenimiento de consistencia lógica en transacciones

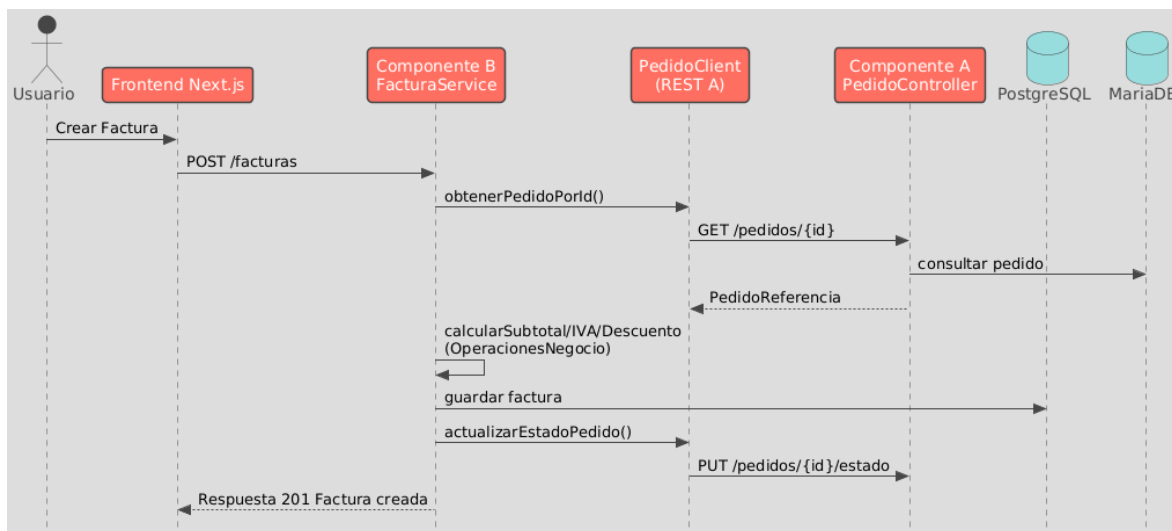


3.5 DIAGRAMA 2 – Modelo de Datos (PlantUML)



- **Explicación del Modelo**
- Un cliente puede tener N pedidos
- Un pedido puede incluir varios productos (N:M)
- Un proveedor puede emitir varias facturas
- Una factura puede incluir varios pedidos → almacenados en texto CSV (pedidoIds)

3.6 DIAGRAMA 3 – Flujo de creación de Factura (Integración A -> B)

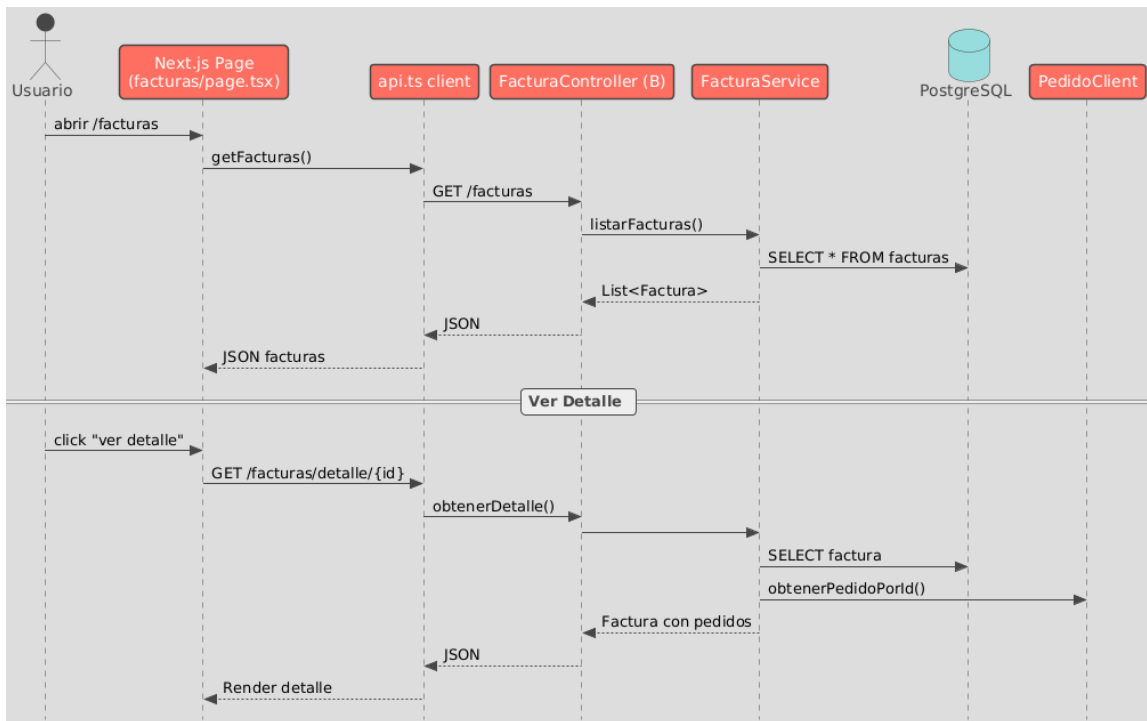


Explicación Técnica

1. El usuario selecciona pedidos desde el frontend
2. Next.js envía la solicitud a /facturas en Componente B
3. B llama a A para obtener detalles completos de cada pedido
4. B calcula subtotal, IVA, descuentos usando Componente C
5. Guarda la factura en PostgreSQL
6. Actualiza estado del pedido en MariaDB mediante A
7. Devuelve la factura al frontend

Esto demuestra **flujos transversales entre microservicios**, un requisito clave del proyecto.

3.7 DIAGRAMA 4 - Flujo de Datos Completo: Backend → Frontend Next.js



Explicación Técnica

1. Next.js (cliente) ejecuta `fetch()` evitando SSR para garantizar datos frescos
2. Los endpoints llamados son:
 - GET /facturas
 - GET /facturas/detalle/{id}
3. La función `handleResponse()` valida JSON y errores
4. El componente React actualiza el estado
5. El frontend renderiza tablas, resúmenes, totales y pedidos asociados
6. La UI es completamente reactiva

3.8 Documentación OpenAPI 3

Ambos microservicios incluyen:

- Schemas generados
- Descripción de DTOs
- Responses codificadas (200, 201, 404, 400)
- Ejemplos y parámetros
- Exploración desde Swagger UI

Microservicio A:

<http://localhost:8080/swagger-ui/index.html>

```
OpenAPI definition
servers:
  - url: http://localhost:8080
    description: Generated server url
paths:
  /pedidos/{id}/estado:
    put:
      tags:
        - pedido-controller
      operationId: cambiarEstado
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
            format: int32
      requestBody:
        content:
          application/json:
            schema:
              type: integer
              format: int32
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: object
      tags:
        - cliente-controller
  /pedidos:
    get:
      tags:
        - cliente-controller
      operationId: listarClientes
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
            format: int32
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: array
  /pedidos:
    post:
      tags:
        - pedido-controller
      operationId: crearPedido
      requestBody:
        content:
          application/json:
            schema:
              type: integer
              format: int32
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: object
  /pedidos/{id}:
    get:
      tags:
        - cliente-controller
      operationId: listarClientes
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
            format: int32
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: array
  /pedidos/{id}:
    delete:
      tags:
        - pedido-controller
      operationId: eliminarPedido
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
            format: int32
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: array
  /pedidos/cliente/{clienteId}:
    get:
      tags:
        - cliente-controller
      operationId: listarPedidosPorCliente
      parameters:
        - name: clienteId
          in: path
          required: true
          schema:
            type: integer
            format: int32
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: array
  /pedidos/cliente/{clienteId}/pendientes:
    get:
      tags:
        - pedido-controller
      operationId: listarPedidosPendientesPorCliente
      parameters:
        - name: clienteId
          in: path
          required: true
          schema:
            type: integer
            format: int32
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: array
  /clientes/{id}:
    put:
      tags:
        - cliente-controller
      operationId: actualizarCliente
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
            format: int32
      requestBody:
        content:
          application/json:
            schema:
              type: integer
              format: int32
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: array
  /clientes/{id}:
    delete:
      tags:
        - cliente-controller
      operationId: eliminarCliente
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
            format: int32
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: array
  /clientes:
    get:
      tags:
        - cliente-controller
      operationId: listarClientes
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: array
  /clientes:
    post:
      tags:
        - cliente-controller
      operationId: crearCliente
      requestBody:
        content:
          application/json:
            schema:
              type: integer
              format: int32
      responses:
        200:
          description: OK
          content:
            */*:
              schema:
                type: array
```

Esquemas

```
Cliente ▾ {  
  id                ▾ integer($int64)  
  nombre            ▾ string  
  correo            ▾ string  
  telefono          ▾ string  
  direccion         ▾ string  
}
```

```
PedidoDTO ▾ {  
  clienteId        ▾ integer($int64)  
  nombre           ▾ string  
  productos        ▾ [ProductoDTO > {...}]  
  subtotal         ▾ number($double)  
  iva              ▾ number($double)  
  descuentoPorcentaje ▾ number($double)  
  descuento        ▾ number($double)  
  total            ▾ number($double)  
  estado          ▾ string  
}
```

```
ProductoDTO ▾ {  
  nombre           ▾ string  
  precio           ▾ number($double)  
  cantidad         ▾ integer($int32)  
}
```

```
Pedido ▾ {  
  id                ▾ integer($int64)  
  clienteId        ▾ integer($int64)  
  nombre           ▾ string  
  subtotal         ▾ number($double)  
  iva              ▾ number($double)  
  descuentoPorcentaje ▾ number($double)  
  descuento        ▾ number($double)  
  total            ▾ number($double)  
  productos        ▾ [Producto > {...}]  
  estado          ▾ string  
}
```

```
Producto ▾ {  
  id                ▾ integer($int64)  
  nombre           ▾ string  
  precio           ▾ number($double)  
  cantidad         ▾ integer($int32)  
}
```

Microservicio B:

<http://localhost:8081/swagger-ui/index.html>

Servers
http://localhost:8081 - Generated server url

proveedor-controller

- GET /proveedores/{id}
- PUT /proveedores/{id}
- DELETE /proveedores/{id}
- GET /proveedores
- POST /proveedores

factura-controller

- PUT /facturas/{id}/anular
- GET /facturas
- POST /facturas
- GET /facturas/detalle/{id}

versión con formato estético

```
{
  "openapi": "3.0.3",
  "info": {
    "title": "Swagger definition",
    "version": "v1",
    "servers": [
      {
        "url": "http://localhost:8081",
        "description": "Generated server url"
      }
    ],
    "paths": {
      "/proveedores/{id}": {
        "get": {
          "tags": [
            "proveedor-controller"
          ],
          "operationId": "actualizar",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "/*/*": {
                  "schema": {
                    "$ref": "#/components/schemas/Proveedor"
                  }
                }
              }
            }
          }
        },
        "put": {
          "tags": [
            "proveedor-controller"
          ],
          "operationId": "actualizar",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "/*/*": {
                  "schema": {
                    "$ref": "#/components/schemas/Proveedor"
                  }
                }
              }
            }
          }
        },
        "delete": {
          "tags": [
            "proveedor-controller"
          ],
          "operationId": "eliminar",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "/*/*": {
                  "schema": {
                    "$ref": "#/components/schemas/Proveedor"
                  }
                }
              }
            }
          }
        }
      },
      "/proveedores": {
        "get": {
          "tags": [
            "proveedor-controller"
          ],
          "operationId": "listar",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "/*/*": {
                  "schema": {
                    "$ref": "#/components/schemas/Proveedor"
                  }
                }
              }
            }
          }
        },
        "post": {
          "tags": [
            "proveedor-controller"
          ],
          "operationId": "crear",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "/*/*": {
                  "schema": {
                    "$ref": "#/components/schemas/Proveedor"
                  }
                }
              }
            }
          }
        }
      },
      "/facturas/{id}/anular": {
        "put": {
          "tags": [
            "factura-controller"
          ],
          "operationId": "anular",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "/*/*": {
                  "schema": {
                    "$ref": "#/components/schemas/Factura"
                  }
                }
              }
            }
          }
        }
      },
      "/facturas": {
        "get": {
          "tags": [
            "factura-controller"
          ],
          "operationId": "listar",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "/*/*": {
                  "schema": {
                    "$ref": "#/components/schemas/Factura"
                  }
                }
              }
            }
          }
        },
        "post": {
          "tags": [
            "factura-controller"
          ],
          "operationId": "crear",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "/*/*": {
                  "schema": {
                    "$ref": "#/components/schemas/Factura"
                  }
                }
              }
            }
          }
        }
      },
      "/facturas/detalle/{id}": {
        "get": {
          "tags": [
            "factura-controller"
          ],
          "operationId": "detalle",
          "parameters": [
            {
              "name": "id",
              "in": "path",
              "required": true,
              "schema": {
                "type": "integer",
                "format": "int64"
              }
            }
          ],
          "responses": {
            "200": {
              "description": "OK",
              "content": {
                "/*/*": {
                  "schema": {
                    "$ref": "#/components/schemas/FacturaDetalle"
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Esquemas

```
Proveedor {
  id          integer($int64)
  nombre      string
  correo      string
  telefono    string
  direccion   string
}
```

```
FacturalInput {
  proveedorId integer($int64)
  pedidosIds  [ > [...]]
  montoManual number($double)
  descuentoPorcentaje number($double)
}
```

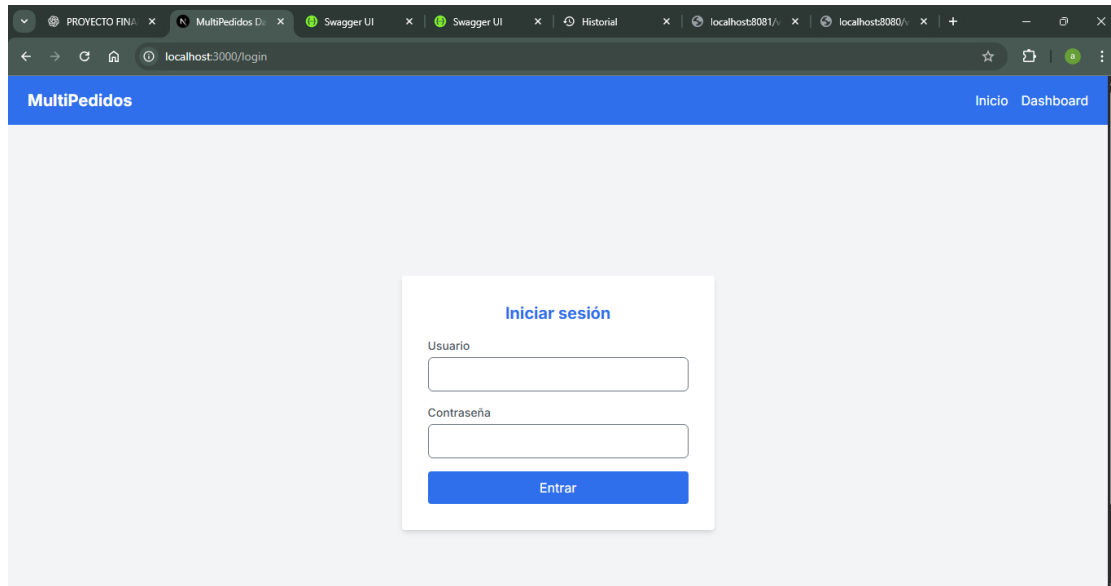
```
Factura {
  id          integer($int64)
  proveedorId integer($int64)
  monto       number($double)
  fecha       string($date)
  subtotal    number($double)
  iva         number($double)
  descuentoPorcentaje number($double)
  descuento   number($double)
  totalFactura number($double)
  estado      string
  motivoAnulacion string
  pedidoIds   string
  pedidos     [PedidoReferencia > {...}]
}
```

```
PedidoReferencia {
  id          integer($int64)
  pedidoId    integer($int64)
  clienteId   integer($int64)
  nombre      string
  subtotal    number($double)
  iva         number($double)
  descuento   number($double)
  total       number($double)
}
```

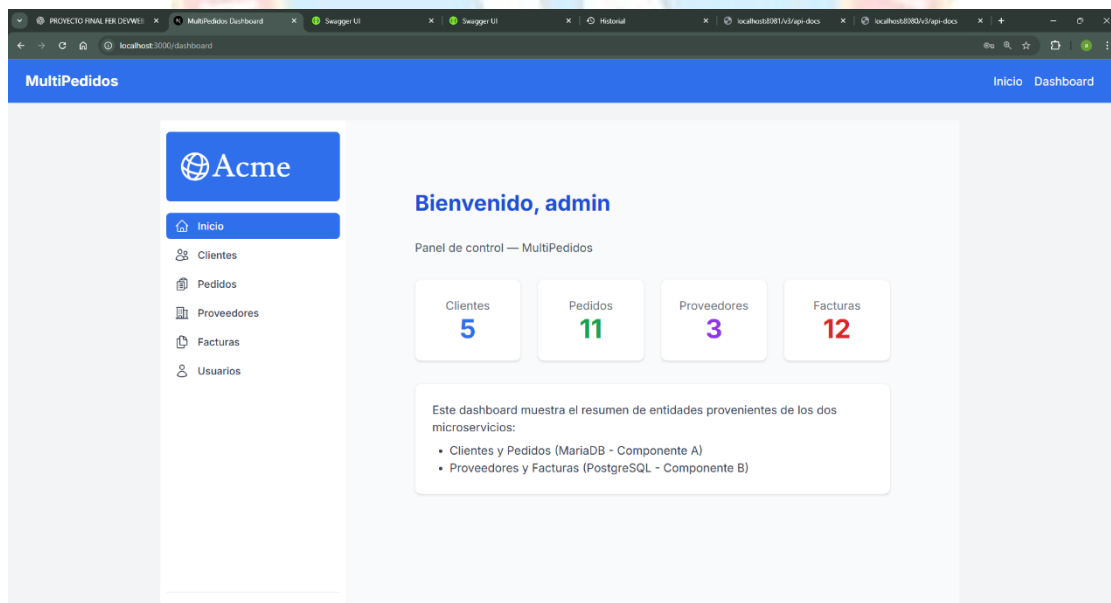
Esto demuestra conocimiento completo en estandarización REST.

3.9 Aplicación Web

3.9.1 Login



3.9.2 Dashboard



3.9.3 Componente Clientes

MultiPedidos Inicio Dashboard

Clientes

Nuevo cliente

Nombre

Correo

Teléfono Opcional

Dirección Opcional

Guardar

Listado de clientes

ID	Nombre	Correo	Teléfono	Dirección	Acciones
1	Fernando	edgarfernandoramos19@gmail.com	46895692	barrio santa cruz	Editar Ver Eliminar
2	Edgar	edgar@gmail.com	46895692	la libertad peten	Editar Ver Eliminar
3	Ricarby	tilin@gmail.com	456789123	La flor del quiche	Editar Ver Eliminar
4	Carlos	carlos@gmail.com	46895692	Poptun, Peten	Editar Ver Eliminar
5	Javier	hugo@gmail.com	46895692	Poptun, Peten	Editar Ver Eliminar

3.9.4 Componente Pedidos

MultiPedidos Inicio Dashboard

Pedidos

Nuevo pedido

Cliente

Dirección del pedido

Productos

Nombre Precio (Q) Cantidad

+ Agregar producto

Guardar pedido

ID	Cliente	Dirección pedido	Productos	Detalles	Estado	Acciones
8	Fernando	Dolores	• Carne — Q350.00 × 0	Subtotal: Q 350.00 IVA (12%): Q 42.00 Total: Q 392.00	Anulado	Eliminar
9	Ricarby	Poptun	• Lomo Cerdo — Q30.00 × 30 • Costilla Cerdo — Q35.00 × 40 • Chorizo — Q15.00 × 40	Subtotal: Q 2900.00 IVA (12%): Q 348.00 Total: Q 3248.00	Facturado	Eliminar
10	Edgar	San Luis	• Aceite Oliva — Q25.00 × 20 • Margarina Mirasol — Q30.00 × 40 • Arroz Macarena — Q5.00 × 40 • Frijol Ipala — Q4.00 × 50	Subtotal: Q 2100.00 IVA (12%): Q 252.00 Total: Q 2352.00	Facturado	Eliminar
11	Carlos	Dolores, Peten	• Aceite Virgen — Q20.00 × 50	Subtotal: Q 1940.00	Entendido	Eliminar

3.9.5 Componente Proveedores

MultiPedidos

Inicio Dashboard

Proveedores

Nuevo proveedor

Nombre

Correo

Teléfono (opcional)

Dirección (opcional)

Guardar

Listado de proveedores

ID	Nombre	Correo	Teléfono	Dirección	Acciones
3	COCA COLA	cocacola@gmail.com	12345678	Coca cola city	Editar Eliminar
4	CARNES FRESCAS DEYCRHS	carnesfresas@gmail.com	46895692	Dolores, Peten	Editar Eliminar
5	TILIN Y ASOCIADOS	tilinero@gmail.com	12345678	Barrio el Porvenir, Poptun, Peten	Editar Eliminar

3.9.6 Componente Facturas

MultiPedidos

Inicio Dashboard

Facturas

Nueva Factura

Proveedor

Selecciona un proveedor

Cliente (para ver pedidos pendientes)

-- Sin cliente seleccionado --

Si seleccionas un cliente, podrás marcar sus pedidos pendientes para incluirlos en la factura.

Pedidos pendientes del cliente

Selecciona un cliente para ver sus pedidos pendientes.

Monto manual (opcional)

Si no seleccionas pedidos, indica un monto manual

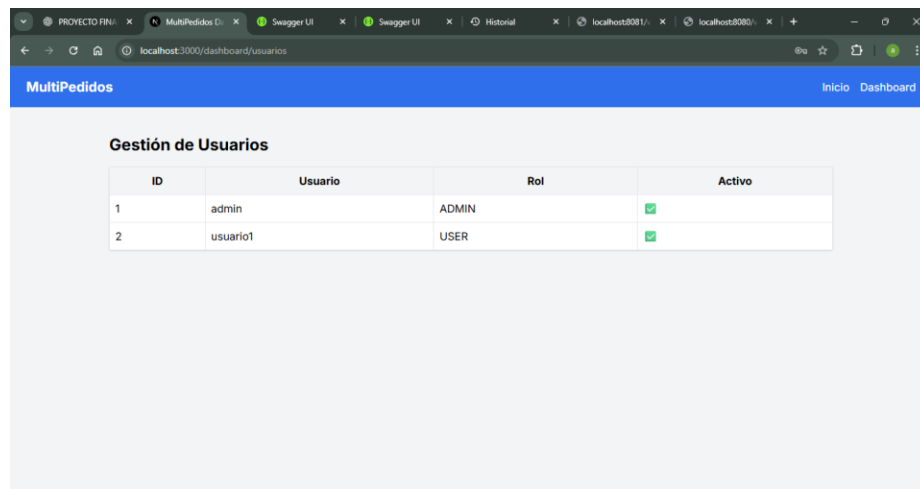
Descuento (%) (opcional)

Crear Factura

Listado de facturas

ID	Proveedor	Subtotal	IVA	Descuento	Total Final	Fecha	Tipo	Estado	Acciones
2	COCA COLA	Q 0.00	Q 0.00	Q 0.00	Q 0.00	2025-11-08	Manual	ANULADA	Ver detalle
1	1	Q 0.00	Q 0.00	Q 0.00	Q 0.00	2025-11-08	Manual	ANULADA	Ver detalle
3	1	Q 560.00	Q 67.20	Q 0.00	Q 627.20	2025-11-08	Manual	ANULADA	Ver detalle
5	CARNES FRESCAS DEYCRHS	Q 3248.00	Q 389.76	Q 0.00	Q 3637.76	2025-11-08	Manual	ACTIVA	Ver detalle Amular
6	TILIN Y ASOCIADOS	Q 4972.80	Q 596.74	Q 278.48	Q 5291.06	2025-11-12	Manual	ACTIVA	Ver detalle Amular
4	COCA COLA	Q 382.00	Q 47.04	Q 8.76	Q 430.26	2025-11-08	Manual	ANULADA	Ver detalle
7	TILIN Y ASOCIADOS	Q 2352.00	Q 282.24	Q 26.34	Q 2607.90	2025-11-12	Manual	ACTIVA	Ver detalle Amular
8	CARNES FRESCAS DEYCRHS	Q 1960.00	Q 235.20	Q 21.95	Q 2173.25	2025-11-13	Manual	ACTIVA	Ver detalle Amular
9	CARNES FRESCAS DEYCRHS	Q 0.00	Q 0.00	Q 0.00	Q 0.00	2025-11-13	Manual	ANULADA	Ver detalle
10	CARNES FRESCAS DEYCRHS	Q 0.00	Q 0.00	Q 0.00	Q 0.00	2025-11-13	Manual	ACTIVA	Ver detalle Amular
11	CARNES FRESCAS DEYCRHS	Q 5600.00	Q 672.00	Q 313.60	Q 5958.40	2025-11-13	Manual	ACTIVA	Ver detalle Amular
12	TILIN Y ASOCIADOS	Q 4144.00	Q 497.28	Q 232.08	Q 4409.22	2025-11-14	Manual	ACTIVA	Ver detalle Amular

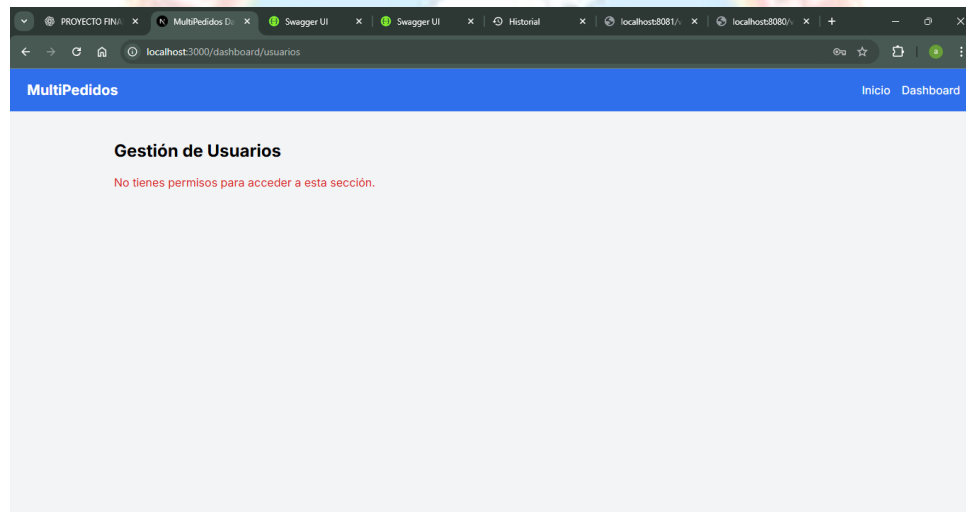
3.9.7 Componente Usuarios – Vista Admin



The screenshot shows a web browser window with the URL `localhost:3000/dashboard/usuarios`. The page has a blue header with the text "MultiPedidos" and navigation links "Inicio" and "Dashboard". The main content area is titled "Gestión de Usuarios" and contains a table with the following data:

ID	Usuario	Rol	Active
1	admin	ADMIN	☒
2	usuario1	USER	☒

3.9.8 Componente Usuarios – VistaUser



4 CONCLUSIONES

Este proyecto cumple todo lo solicitado:

- Microservicios independientes
- Bases de datos distintas
- Integración REST entre sistemas
- Frontend moderno
- Cálculos centralizados en un módulo Maven
- Documentación OpenAPI
- Arquitectura profesional y escalable

Adicionalmente, se agregó un componente extra de autenticación para reforzar seguridad y modularidad.

